Jarvis Build Guide: Online/Local Hybrid

Oct 24, 2026 · @Naresh

How to use this guide

Build Jarvis one phase at a time, giving each phase's prompt to your AI coding assistant, and move on only when that phase passes its "done when" test.

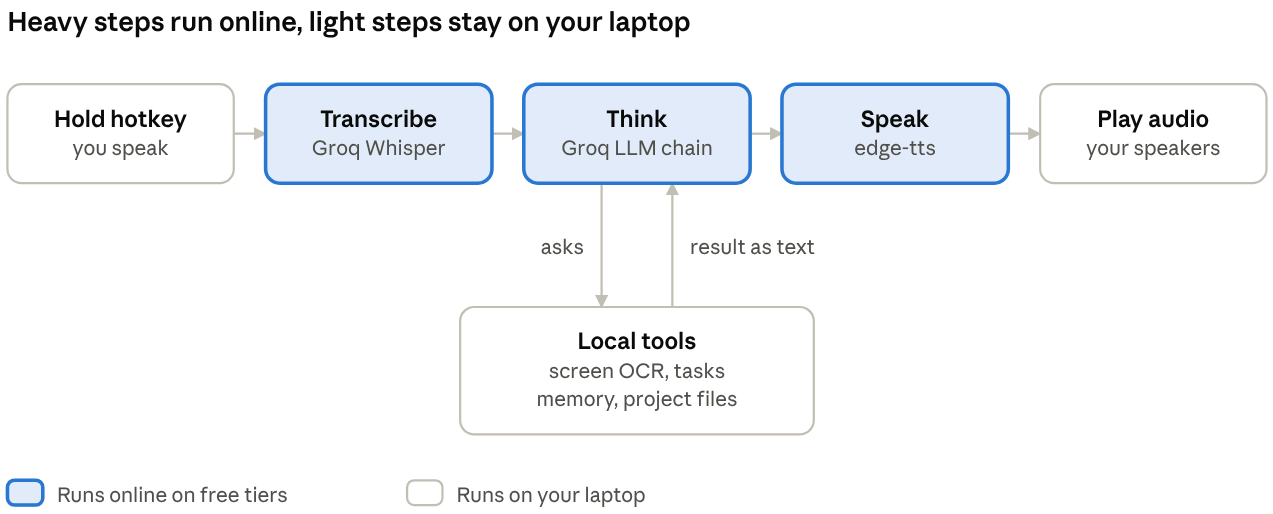
- Paste the project brief (next section) at the top of every new session so the assistant keeps the whole design in mind.
- Run the code yourself after each phase. If something fails, paste the exact error back before going further.
- Keep API keys in a `.env` file on your laptop. Never paste a key into a chat or commit it anywhere.
- Model names and free-tier limits change often. The prompts tell the assistant to read each provider's current model list instead of hard-coding names, and you should check your own limits in each provider's console.
- Times and speeds in this guide are estimates. Measure on your laptop and adjust.

The design on one page

Anything that needs a big model runs online on free API tiers, and anything light runs on your laptop, so your CPU, GPU and RAM stay free for your own work.

Job	Runs	Tool
Brain and reasoning	Online	Groq first, then Gemini, Cerebras, OpenRouter free models
Speech-to-text	Online	Groq Whisper large-v3-turbo
Voice generation	Online	edge-tts, with Piper as a local backup
Image understanding	Online, only on request	Cloud vision model
Hold-to-talk hotkey, mic, playback	Local	pynput or keyboard, sounddevice

Job	Runs	Tool
Screenshot (view-only)	Local	mss
Screen text	Local	RapidOCR
Tasks, memory, reminders	Local	SQLite, APScheduler
Phone alerts	Local call to a free service	ntfy.sh
Project folder reader	Local, read-only	Plain file reads locked to one root



one voice request · 5 steps, 1 tool loop

Only transcription, thinking and voice generation leave your laptop. Screen text, tasks and files are handled locally, and the brain gets their results as plain text.

Paste this brief at the top of every phase session:

```
PROJECT BRIEF (paste at the start of every session)
I am building "Jarvis", a personal voice assistant for Windows 11 (i5-14450HX, 16 GB RAM, RTX 4050 6 GB). Rule: anything needing a big model runs online through free API tiers; anything light runs locally. Do not run any local LLM or local Whisper, and do not use the GPU.
Online: LLM brain through a fallback chain (Groq first, then Gemini, Cerebras, OpenRouter free models), speech-to-text (Groq Whisper large-v3-turbo), voice generation (edge-tts), image understanding (cloud vision model, only when I ask).
Local: hold-to-talk hotkey, mic recording, silence trimming, screenshot capture (view-only, no mouse or keyboard control), OCR (RapidOCR), SQLite tasks and memory, APScheduler reminders, ntfy.sh phone alerts, project-folder reader (read-only, locked to one root), audio playback, tray icon.
Goals: (1) task and deadline reminders, (2) coding mentor that reads my screen on demand and my project folder, (3) general assistant (weather, reasoning), (4) interactive teacher for new topics such as ESP32 with a small project.
Personality: calm, warm, curious, speaks like a human mentor, short spoken answers, no bullet points or headers, asks one question at a time, says "I'm not sure" when unsure.
Constraints: Python 3.11+, near-zero CPU when idle, every external service behind a swappable interface, secrets only in .env, log every tool action, confirm before anything irreversible.
Work in small steps: show me the files you create, how to run them, and how to test them. Ask me before changing earlier phases.
```

Phase 0: Setup

Goal: a clean Python project, your accounts and keys in place, and a health check that proves your mic, speaker and internet work before any real feature exists.

Do this yourself first:

- Install Python 3.11 or newer and VS Code (or your IDE).
- Create free accounts and API keys at Groq, Google AI Studio (Gemini), Cerebras and OpenRouter.
- Pick a long random topic name for ntfy.sh and subscribe to it in the ntfy phone app. Treat the topic name like a password, because anyone who knows it can send you notifications.

Prompt:

```
[Paste the PROJECT BRIEF here]

PHASE 0: Set up the project skeleton.
1. Create a Python project called jarvis with a src layout, a virtual
environment, requirements.txt, and a README with run instructions for Windows
11 PowerShell.
2. Create config.yaml holding the provider chain (ordered list of providers,
each with base_url, env var name for the key, and a list of preferred
models), the home city, timezone Asia/Kolkata, the hotkey, the project root
path, and the ntfy topic env var name. Do not hard-code model names; leave
them as placeholders I fill in from each provider's console.
3. Create .env.example listing every key name (GROQ_API_KEY, GEMINI_API_KEY,
CEREBRAS_API_KEY, OPENROUTER_API_KEY, NTFY_TOPIC) and make sure .env is in
.gitignore.
4. Add structured logging to a rotating file under logs/ and to the console.
5. Add a command `python -m jarvis.doctor` that checks: Python version, each
key present (never print the key), internet reachable, each provider
reachable with a tiny test call, default microphone and speaker found, and
prints a clear pass/fail table.
Show me every file, then tell me how to run the doctor command.
```

Done when: `python -m jarvis.doctor` shows green for Python, internet, mic, speaker and at least one provider key.

Phase 1: Brain, fallback chain and persona

Goal: a text chat in the terminal that talks like a calm mentor and keeps working when one provider fails. This phase decides how Jarvis feels, so spend time here.

Before you start, open each provider's console, read the current model list and your rate limits, and note the strongest model, a second large model, and a fast small model. Ask the assistant to fill config.yaml with those.

Prompt:

```
[Paste the PROJECT BRIEF here]

PHASE 1: Build the brain.
1. Create a provider abstraction using the OpenAI-compatible chat API for
Groq, Gemini, Cerebras and OpenRouter. Read the ordered chain from
config.yaml.
2. On a 429, 5xx, timeout or retired-model error, fall through to the next
model or provider, with a short cooldown so a throttled model is skipped for
a while. Log which provider answered each turn.
3. At startup, fetch each provider's live model list and warn me if a
configured model no longer exists, instead of crashing.
4. Support streaming responses (yield text as it arrives) and tool calling,
with a simple registry where a tool is a Python function plus a JSON schema.
5. Add two tiers: fast (small talk, reminders, simple lookups) and strong
(reasoning, teaching, debugging). Use a simple rule-based router first; the
model decides nothing about routing yet.
6. Put the personality in persona.md and load it as the system prompt. Rules:
this is spoken aloud, so 1 to 4 short sentences by default, no bullets, no
headers, no numbered steps, one idea at a time, ask at most one question
back, use everyday analogies, admit uncertainty, never say "as an AI".
Include five short example exchanges (small talk, a technical question, being
unsure, a reminder request, a learner who is confused) written in the exact
voice I want: warm, calm, curious, a little witty.
7. Add a terminal chat loop to test it, with a /strong and /fast switch and a
/provider command that shows which one answered.
Show me the files and how to run the chat.
```

Done when: you can chat in the terminal, replies sound spoken instead of like a list, and removing your Groq key makes it fall through to the next provider without an error message.

Phase 2: Tasks and reminders

Goal: you say a task and a deadline, and plain code, not the AI model, reminds you a day before, an hour before and so on, then keeps nudging until you mark it done.

Prompt:

```
[Paste the PROJECT BRIEF here]

PHASE 2: Build the task and reminder engine. Reminders must work without the
LLM or internet running.
1. SQLite database with a tasks table (id, title, notes, deadline,
importance, status, created_at, completed_at) and a reminders_sent table so
nothing fires twice.
2. LLM tools: add_task, list_tasks(range: today, week, overdue, all),
complete_task, snooze_task, delete_task. Deleting needs spoken confirmation.
3. The LLM only converts my words into structured fields. Use Asia/Kolkata.
Always read the parsed deadline back to me in plain language ("Friday at 5
PM, right?") before saving when the time was vague.
4. Scheduler (APScheduler with a SQLite job store) creates reminders for
every task: 1 day before, 1 hour before, 10 minutes before, at the deadline,
then repeat every 30 minutes (configurable) while overdue and not done. Skip
any reminder time already in the past when the task is created. Tasks marked
important get an extra reminder 3 hours before.
5. On startup, find reminders that were missed while the laptop was off and
announce them immediately in one grouped message.
6. Delivery: a spoken or console message on the laptop, plus an ntfy.sh push
to my phone (HTTP POST to the topic in .env). If ntfy fails, log it and keep
going.
7. When I ask "what's pending?", "anything today?" or "what did I miss?", the
answer must come from list_tasks reading the database, never from memory.
Phrase it naturally ("Two things today. The report is due at five...").
8. A configurable morning briefing time that summarizes today's and overdue
tasks.
9. Unit tests for the reminder-time calculation, including tasks due in under
an hour, tasks already overdue, and timezone handling.
```

Done when: you add a task due in about three hours and see the correct reminders scheduled, restart the app and they persist, a test reminder reaches your phone, and "what's pending?" lists the right tasks.

Phase 3: Voice in

Goal: hold a key, speak, release, and see an accurate transcript within a second or two, with no CPU use while idle.

I'd avoid Ctrl+Shift+Tab because Chrome and VS Code already use it. Ctrl+Alt+Space is a safer default, and the prompt makes it configurable.

Prompt:

```
[Paste the PROJECT BRIEF here]

PHASE 3: Add hold-to-talk speech input.
1. A global hotkey listener (pynput or keyboard) that works while any other
window is focused. Default Ctrl+Alt+Space, configurable in config.yaml.
Behavior: hold to record, release to send.
2. Record from the default microphone with sounddevice at 16 kHz mono, only
while the key is held. Nothing is recorded or processed when idle.
3. Optional Silero VAD to trim leading and trailing silence before upload.
4. Send the audio to Groq speech-to-text using the whisper-large-v3-turbo
model through the OpenAI-compatible audio transcriptions endpoint. Pass
language=en by default and a short prompt containing my frequently used names
and technical terms (read from config) to improve spelling.
5. Handle failures: too-short audio is ignored, network errors retry once,
and then the console says it could not hear me.
6. Print the transcript and feed it into the Phase 1 brain instead of typed
text. Keep typed chat working.
7. Add timing logs for record length, upload and transcription time.
8. Keep CPU use low: cap worker threads, run the process at below-normal
priority, and make sure the listener thread sleeps when idle.
9. Make the speech-to-text provider a swappable interface, with an optional
local faster-whisper implementation that is disabled by default.
```

Done when: holding the hotkey in any window and speaking prints the right transcript, the delay after release is short enough to feel natural, and Task Manager shows near-zero CPU while idle.

Phase 4: Voice out

Goal: Jarvis starts speaking in a calm, soft voice while the answer is still being written, and you can interrupt it.

edge-tts is an unofficial route into Microsoft's neural voices, so it can break without warning. That is why the prompt asks for a local Piper backup behind the same interface.

Prompt:

```
[Paste the PROJECT BRIEF here]

PHASE 4: Add speech output with streaming.
1. A text-to-speech interface with two implementations: edge-tts (primary,
online) and Piper (local backup, used when edge-tts fails or I am offline).
2. Write a small script that lists the available English voices (including
Indian and neutral English options) and plays the same sample sentence in
each so I can audition them. Then add the chosen voice, rate and pitch to
config.yaml. I want calm, soft and unhurried.
3. Stream: take tokens from the brain, split into sentences as they arrive,
synthesize each sentence as soon as it is complete, and queue the audio so
playback starts after the first sentence without waiting for the full answer.
4. Clean text before speaking: strip markdown symbols, code blocks, URLs and
emoji. Code and long lists stay on the console and Jarvis says something like
"I've put the code on your screen".
5. Read numbers, dates and times naturally.
6. Barge-in: pressing the hold-to-talk hotkey immediately stops playback and
starts listening.
7. Optional short filler lines ("let me take a look") that play while a slow
tool is running, chosen randomly so they do not repeat.
8. Log time from end of my speech to first audio.
Wire the full loop: hotkey, transcription, brain, voice.
```

Done when: you can hold the key, ask a question, and hear the answer begin within a few seconds, the voice sounds the way you want, and pressing the key mid-sentence cuts it off.

Phase 5: Screen eyes

Goal: when you say "why am I getting this error?", Jarvis takes one screenshot, reads the text on it and explains what is going on, and it can never click or type on your PC.

OCR text is sent to the brain by default, which is faster and uses far fewer tokens than uploading a picture. The image goes to a cloud vision model only when you ask about layout, a diagram or a photo.

Prompt:

```
[Paste the PROJECT BRIEF here]

PHASE 5: Add view-only screen understanding.
1. A look_at_screen tool: capture the primary monitor (or the active window
if easy) on demand using mss. No mouse or keyboard automation code anywhere
in the project.
2. Run RapidOCR on the capture (downscale large screens first), and return
the text with rough positions. Fall back to Tesseract if RapidOCR is
unavailable.
3. Give the OCR text to the brain along with my question. The brain decides
when to call the tool, plus a rule-based fallback: if my sentence contains
phrases like "this error", "my screen", "what's on screen", "this code",
always call it.
4. A second tool, describe_screen_image, that sends the screenshot to a cloud
vision model (check which free vision models are available from my providers)
only when I ask about layout, a diagram, a chart or a photo.
5. Privacy controls: screenshots stay in memory and are never written to
disk; a pause hotkey (default Ctrl+Alt+P) disables capture; a window-title
blocklist in config.yaml (password managers, banking) makes the tool refuse;
a tray icon shows whether screen access is active or paused.
6. Optional, off by default: a rolling OCR history that captures only when
the screen changed, at most every few seconds, and keeps the last few minutes
in RAM so Jarvis can say "this is the third time this error has appeared".
Include a CPU budget test.
7. Say a filler line while capture and OCR run, and log how long each step
takes.
```

Done when: with an error showing in a terminal, you ask about it by voice and Jarvis quotes the right error and explains the likely cause, CPU returns to near zero right after, and the pause hotkey makes it refuse.

Phase 6: Project folder reader and coding mentor

Goal: Jarvis can read your project code, and only that folder, so it can connect an error on screen to the exact file and line and coach you through the fix without changing anything itself.

Prompt:

```
[Paste the PROJECT BRIEF here]

PHASE 6: Add the read-only project reader and the coding mentor behavior.
1. Read the project root path or paths from config.yaml. Tools: list_files,
read_file (with line ranges), search_text. All read-only. There is no write,
delete, move or shell tool.
2. Path safety: resolve every requested path and reject anything outside the
allowed roots, including .. segments, absolute paths, drive letters, and
symlinks or junctions that point outside. Write tests that try to escape the
root.
3. Always skip .env files, key files, .git internals, node_modules, venv
folders and similar, plus any file over a size limit. If I ask for a skipped
file, say it is protected.
4. When the OCR text from the screen mentions a file path and line number or
a stack trace, open that file and show the surrounding lines to the brain
automatically.
5. Mentor behavior in persona.md: explain the cause in plain spoken language
first, then give the fix as one small step at a time. Code snippets go to the
console, not out loud. Never claim to have changed a file.
6. Repeated-error awareness: remember the last few errors I asked about in
this session, and if the same one returns, say so and try a different angle
instead of repeating the same advice.
7. If the problem is unclear, ask me for one specific thing (the full
traceback, the command I ran) rather than guessing.
```

Done when: an error that mentions a file in your project leads Jarvis to name that file and line and explain the cause, asking it to read something outside the project folder is refused, and it never edits a file.

Phase 7: Memory and interactive teaching

Goal: Jarvis remembers you and your projects, and when you say "teach me ESP32" it acts like a mentor who asks, explains one idea, gives you one small thing to try, and reacts to what happens.

Small models and free-tier models can state wrong technical details with confidence, and with hardware that can damage a board. So put real documentation, such as the ESP32 pinout and datasheet notes, into a docs folder yourself, and make Jarvis look pin numbers and voltages up there instead of answering from memory.

Prompt:

```
[Paste the PROJECT BRIEF here]

PHASE 7: Add long-term memory and teaching mode.
1. SQLite tables: facts (things about me and my preferences), projects (name, goal, status, notes, next step), lessons (topic, my level, steps completed, where I got stuck, last session date).
2. Tools: remember_fact, recall, update_project, save_lesson_progress. At the end of each conversation, summarize it into memory. At the start, load the facts and active projects that look relevant.
3. Teaching mode, written into persona.md and the tool logic. Loop: (a) ask what I already know and what I want to build, (b) explain exactly one idea with an everyday analogy in two or three spoken sentences, (c) give me one small thing to do, (d) wait for me to report back, (e) react to what actually happened and adapt: slow down if I am confused, skip ahead if I am not. Never dump a roadmap or a numbered list of steps. Never ask more than one question at a time.
4. A lookup_docs tool that searches a local docs/ folder (markdown or text files I add). For any pin number, voltage, current limit, library name or command, Jarvis must call lookup_docs first and say something like "let me check the pinout". If the docs do not cover it, it says it is not sure instead of guessing.
5. Hardware safety: before wiring steps, Jarvis mentions power, voltage and polarity checks in one calm sentence.
6. First lesson: ESP32 with a small project that grows in stages, so each new idea is needed for the next step: blink an LED, read a button, read one sensor, then connect to Wi-Fi and show the reading. Save progress after every step.
7. Resume: if I say "continue" or "where were we", load the lesson and pick up from the last completed step, briefly recapping first.
```

Done when: "teach me ESP32" starts with a question about your experience instead of a lecture, each reply teaches one idea and asks you to try one thing, and the next day "continue" resumes at the right step.

Phase 8: General tools

Goal: Jarvis handles everyday questions like weather, current facts and quick calculations, and sends hard reasoning to the strong model.

Prompt:

```
[Paste the PROJECT BRIEF here]

PHASE 8: Add general-purpose tools.
1. get_weather: use the Open-Meteo geocoding and forecast APIs (no API key).
Default to the home city in config.yaml, accept another city by voice, and
return a short structured result (now, today's high and low, rain chance,
tomorrow).
2. web_search: use a free search library (DuckDuckGo) and return the top few
results as short titles and snippets. For current facts, the brain must
search instead of answering from memory, and should mention the source
casually in speech ("according to the BBC").
3. current_time and a safe calculator tool, so the model never does
arithmetic or date math by guessing.
4. Tool results are short and structured. The brain turns them into one or
two natural spoken sentences.
5. Improve the router: questions needing multi-step reasoning, explanation or
teaching go to the strong tier, while chit-chat and simple lookups use the
fast tier. Log which tier handled each turn so I can tune it.
6. Add timeouts to every tool so a slow website never freezes the voice loop,
and have Jarvis say so honestly when a tool fails.
```

Done when: asking about the weather, today's date and a current news item all get correct, natural spoken answers, and a slow or failing tool produces a graceful spoken message instead of silence.

Phase 9: Runtime, safety and performance

Goal: Jarvis starts with Windows, sits at near-zero CPU, keeps your reminders working when the internet is down, and cannot do anything risky without asking.

Prompt:

```
[Paste the PROJECT BRIEF here]

PHASE 9: Make it run reliably in the background.
1. Start automatically at login using Windows Task Scheduler (give me the
exact setup steps or a script), with no console window, and a tray icon
(pystray) with Status, Pause listening, Pause screen access, Open logs and
Quit.
2. Run at below-normal priority, cap worker threads, and make sure no loop
polls faster than it needs to. Add a script that samples Jarvis's CPU and RAM
use at idle, during a voice turn, and during a screen read, and prints a
summary.
3. Offline mode: detect no internet quickly. Reminders and the local
scheduler keep working and are announced with the local Piper voice or a
beep. Brain and speech-to-text calls say clearly that I am offline instead of
hanging. Resume normally when the connection returns.
4. Tool risk levels in config: read-only (no confirmation), reversible
(spoken confirmation), irreversible (explicit confirmation phrase). Deleting
tasks and anything that sends data outside the laptop counts as irreversible.
5. An action log that records every tool call with time, arguments and
result, viewable from the tray menu.
6. Daily backup of the SQLite database, keeping the last 7 copies.
7. Reload config.yaml without restarting, and surface startup errors as one
spoken sentence plus a log entry.
```

Done when: after a reboot Jarvis is running without you touching it, the hotkey works, an idle reading in Task Manager stays near zero, a reminder fires with the Wi-Fi turned off, and the action log shows what it did.

Tuning checklist and common problems

Most problems fall into a handful of causes, and the fix is usually a config change or a prompt edit, not new code.

Problem	Likely cause	What to try
Answers sound like lists or lectures	Persona too weak, or the model is ignoring it	Add more example exchanges to persona.md, shorten the style rules, test on the strong tier

Problem	Likely cause	What to try
Slow first word	Waiting for the whole answer before speaking, or a cold connection	Confirm sentence-level streaming, reuse HTTP sessions, trim silence before upload
Rate-limit errors (429)	Free-tier daily or per-minute cap hit	Check your limits in each provider's console, send small talk to the fast tier, add another provider
A model stops working	Provider retired or renamed it	Run the startup model check, update config.yaml
Hotkey does nothing in some windows	Global hotkey libraries often cannot see keys pressed in windows running as administrator	Test in a normal window first, then decide whether to run Jarvis elevated
Names or terms transcribed wrong	No spelling hint	Add them to the transcription prompt list in config
Voice sounds robotic	Wrong voice, rate or pitch	Re-run the voice audition and lower the rate slightly
Screen answers miss the error	OCR misread small text	Increase the capture scale, or ask Jarvis to look at the terminal full-screen
Teaching answers include wrong technical details	Model guessed instead of checking docs	Add the missing documentation to docs/ and keep the lookup-first rule
Laptop feels slower	Something is polling or running at normal priority	Run the resource sampling script and look for loops without sleeps

Later upgrades

Once the nine phases work and feel good, these are the next layers, each as its own phase with its own prompt.

- Wake word instead of a hotkey, once you have measured the idle CPU cost.

- Camera or phone photo input, so Jarvis can check a breadboard wiring photo.
- A bigger local docs library (datasheets, your own notes) searched with embeddings.
- Email and Telegram alerts alongside ntfy.
- A smarter router that picks the model by question difficulty and remaining free-tier budget.
- An optional local fallback model, only if outages or limits start to bother you.
- Always-on error history for repeated-error detection, if you decide the CPU cost is acceptable.